Internet stvari (IoT)
(3OER6A01)

Arhitektura mikrokontrolera
ATmega2560 (2.deo)

Predavanja

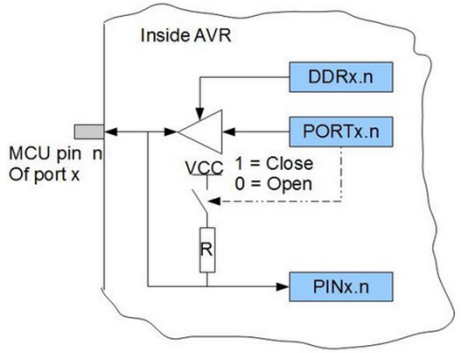
Sadržaj

- Portovi
- Brojači i tajmeri
- A/D i D/A konverzija
- PWM

U/I portovi

	DDRx = 0	DDRx = 1
PORTx = 0	Ulaz → PINx (tri-state)	Izlaz je 0
PORTx = 1	Ulaz → PINx (pull-up)	Izlaz je 1

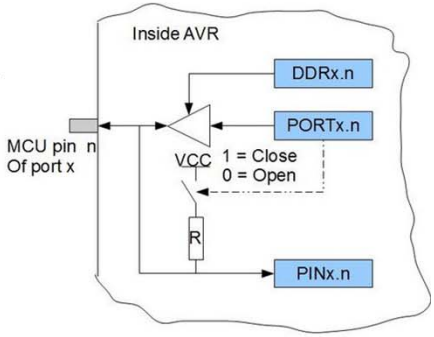
- Ulazno-izlazni (U/I) portovi su interfejs mikrokontrolera ka spoljašnjim uređajima.
- Portovi su hardverske komponente koje čine pinovi (mogu se nezavisno konfigurisati kao ulazni ili izlazni) i registri (koji upravljaju smerom i stanjem pinova).
- ATmega alocira tri U/I memorijske adrese za svaki od portova:
  - Data Register – PORTx (read/write),
  - Data Direction Register – DDRx (read/write) i
  - Port Input Pins – PINx (read only).
- Dodatno, Pull-up Disable – PUD bit u MCUCR registru onemogućuje pull-up funkciju za sve pinove na svim portovima, ako je setovan.



MCUCR – MCU Control Register

Bit	7	6	5	4	3	2	1	0	
0x35 (0x55)	JTD	–	–	PUD	–	–	IVSEL	IVCE	MCUCR
Read/Write	R/W	R	R	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Registri portova



PORTA – Port A Data Register

Bit	7	6	5	4	3	2	1	0	
0x02 (0x22)	PORTA7	PORTA6	PORTA5	PORTA4	PORTA3	PORTA2	PORTA1	PORTA0	PORTA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

DDRA – Port A Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x01 (0x21)	DDA7	DDA6	DDA5	DDA4	DDA3	DDA2	DDA1	DDA0	DDRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

PINA – Port A Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x00 (0x20)	PINA7	PINA6	PINA5	PINA4	PINA3	PINA2	PINA1	PINA0	PINA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

Programsko upravljanje portovima

```
#include <avr/io.h>
int main(void)
{
  //set all pins as output
  DDRA = 0b11111111; // 0xFF
  //write ones and zeros to pins
  PORTA = 0b10101010; // 0xAA
  return 0;
}
```

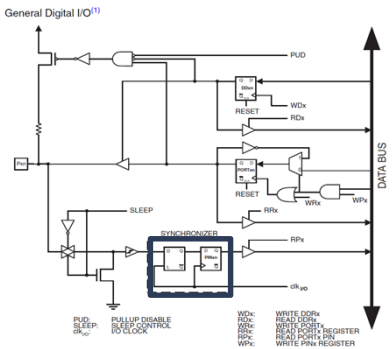
```
#include <avr/io.h>
int main(void)
{
  uint8_t x;
  // set lower nibble to input
  // this operation clears bits 0,1,2,3 of DDRA
  DDRA &= ~((1<<PA0) | (1<<PA1) | (1<<PA2) | (1<<PA3));
  // setting pull-ups
  PORTA |= (1<<PA0) | (1<<PA1) | (1<<PA2) | (1<<PA3);
  // set higher nibble to output
  /* bit shift “<<”operation and logical OR "|"
  used to set bits 4,5,6,7 to “1” */
  DDRA |= (1<<PA4) | (1<<PA5) | (1<<PA6) | (1<<PA7);
  // write value to higher nibble
  PORTA |= (1<<PA4) | (1<<PA5) | (1<<PA6) | (1<<PA7);
  // read value from lower nibble
  // and filter out high nibble
  x = (0b00001111) & PINA;
  return 0;
}
```

Programsko upravljanje portovima

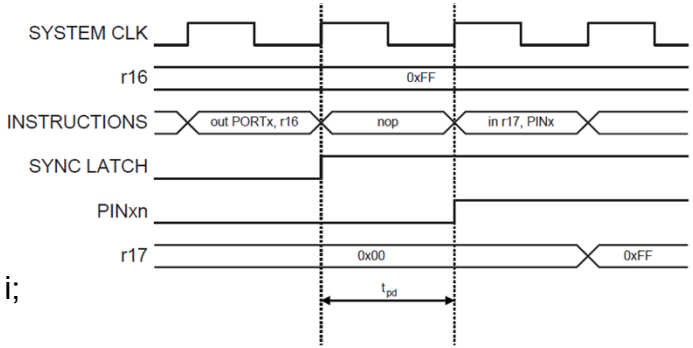
```
#include <avr/io.h>
//define ports and pins
#define INDDR DDRA
#define INPORT PORTA
#define INPIN PINA
#define OUTDDR DDRB
#define OUTPORT PORTB
#define BUTTON PA0
#define LED PB0

int main(void)
{
    uint8_t x;
    // set BUTTON pin as input
    INDDR &= ~(1 << BUTTON);
    // setting internal pull-up for button
    INPORT |= (1 << BUTTON);
    // seting LED pin as output
    OUTDDR |= (1 << LED);
    // turning LED1 ON
    OUTPORT |=  (1 << LED);
    // read BUTTON state
    // and test with mask
    x = (1 << BUTTON) & INPIN;
    return 0;
}
```

```
#include <avr/io.h>
//define ports and pins
#define INDDR DDRA
#define INPORT PORTA
#define INPIN PINA
#define OUTDDR DDRB
#define OUTPORT PORTB
#define BUTTON PA0
#define LED PB0

#define LEDON OUTPORT |= (1 << LED)
#define BUTTONIN (1 << BUTTON) & INPIN

int main(void)
{
    uint8_t x;
    // set BUTTON pin as input
    INDDR &=~(1<<BUTTON);
    // setting internal pull-up for button
    INPORT |=(1<<BUTTON);
    // seting LED pin as output
    OUTDDR |=(1<<LED);
    // turning LED ON
    LEDON;
    // read BUTTON state
    x=BUTTONIN;
    return 0;
}
```

Čitanje softverski postavljene vrednosti



Prilikom čitanja vrednosti pina dodeljene softverom, mora se umetnuti instrukcija **nop**. Instrukcija **out** postavlja signal „SYNC LATCH“ na pozitivnu ivicu takta. U ovom slučaju, kašnjenje t_{pd} kroz sinhronizator (PINxn registarski bit i prethodni leč) je jedan period sistemskog takta.




```
...
; Define pull-ups and set outputs high
; Define directions for port pins
ldi r16,(1<<PB7)|(1<<PB6)|(1<<PB1)|(1<<PB0)
ldi r17,(1<<DDB3)|(1<<DDB2)|(1<<DDB1)|(1<<DDB0)
out PORTB,r16
out DDRB,r17
; Insert nop for synchronization
nop
; Read port pins
in r16,PINB
...
```

Primer asemblerskog koda

```
unsigned char i;
...
/* Define pull-ups and set outputs high */
/* Define directions for port pins */
PORTB = (1<<PB7)|(1<<PB6)|(1<<PB1)|(1<<PB0);
DDRB = (1<<DDB3)|(1<<DDB2)|(1<<DDB1)|(1<<DDB0);
/* Insert nop for synchronization*/
__no_operation();
/* Read port pins */
i = PINB;
...
```

Primer C koda

Nepovezani pinovi

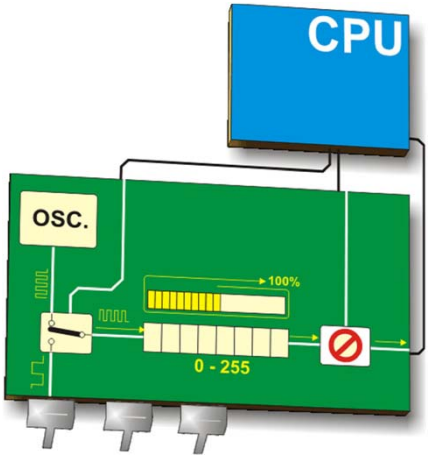
- Ako su neki pinovi neiskorišćeni, preporučuje se da se osigura da ti pinovi imaju definisan naponski nivo.
- Plutajuće ulaze treba izbegavati kako bi se smanjila potrošnja struje u svim režimima gde su digitalni ulazi omogućeni.
- Najjednostavniji metod za obezbeđivanje definisanog nivoa neiskorišćenog pina jeste omogućavanje internog pull-up-a. U ovom slučaju, pull-up će biti onemogućen tokom resetovanja. Ako je važna niska potrošnja energije tokom resetovanja, preporučuje se korišćenje eksternog pull-up-a ili pull-down-a.
- Povezivanje neiskorišćenih pinova direktno na VCC ili GND se ne preporučuje, jer to može prouzrokovati prekomerne struje ako se pin slučajno konfiguriše kao izlaz.

Alternativne funkcije portova

- **General Purpose I/O (GPIO) –** Svih 54 digitalnih pinova mogu se koristiti kao standardni digitalni ulazi ili izlazi.
- **PWM Output –** 15 digitalnih pinova može se konfigurisati kao PWM izlazi, što omogućava kontrolu brzine motora, osvetljenosti LED dioda itd.
- **UART** – 4 hardverska serijska porta (UART) za komunikaciju sa drugim uređajima. Serijski port 1 (pinovi 18 i 19), Serijski port 2 (pinovi 16 i 17) i Serijski port 3 (pinovi 14 i 15).
- **SPI** – SPI magistrala (MISO, MOSI, SCK, SS) omogućava brzu komunikaciju sa raznim perifernim uređajima poput memorijskih kartica, displeja i senzora.
- **Prekidi** – Mnogi digitalni pinovi mogu biti konfigurisani kao pinovi za prekide, što omogućava mikrokontroleru da reaguje na spoljne događaje.
- **Analogni ulazi** – 16 pinova se može koristiti kao analogni ulazi, pružajući 10-bitnu rezoluciju (0-1023) za merenje analognih nivoa napona.
- **TWI/I2C** – Pinovi SDA i SCL se koriste za TWI (Two-Wire Interface), takođe poznat kao I^2C (Inter-Integrated Circuit), što je još jedan komunikacioni protokol.
- **AREF** – AREF pin se koristi za podešavanje referentnog napona za analogne ulaze (omogućava skaliranje ulaznog opsega).

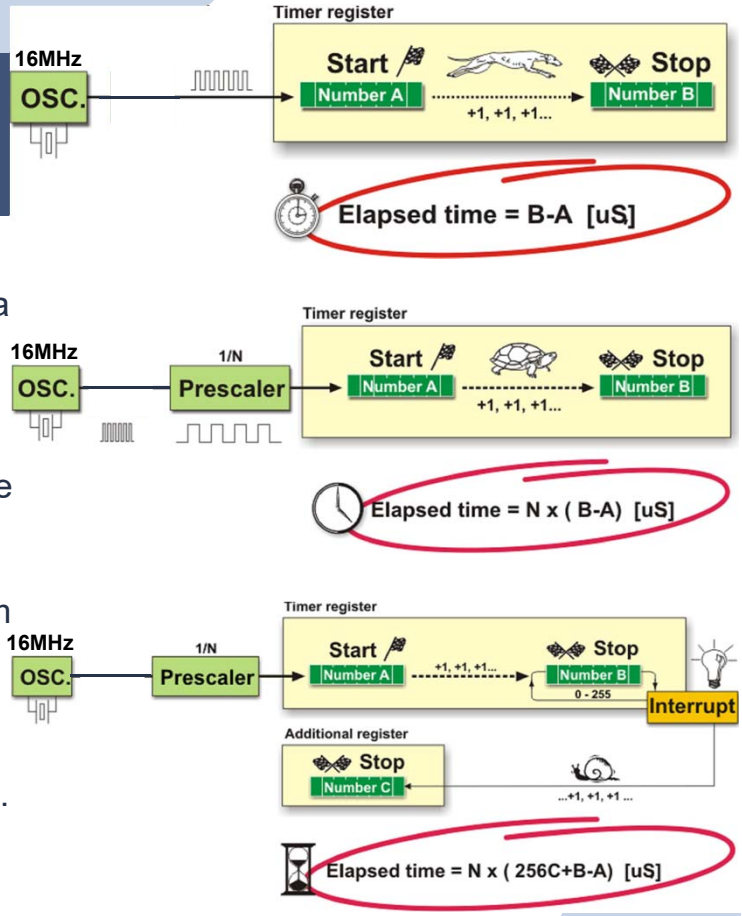
Tajmeri i brojači

- **Brojač** je uređaj koji beleži broj pojavljivanja nekog događaja (obično broji prelaske sa 0 na 1 ili obrnuta na nekom pinu). Izvor je eksterni signal.
- **Tajmer** meri vremenske intervale (broji unapred) ili odbrojava unazad i definiše kašnjenje (obično broji mašinske cikluse). Izvor je interni klok.
- Tajmeri se koriste i za PWM (Pulse Width Modulation – modulacija širine impulsa), tehniku za kodiranje informacija ili kontrolu snage koja se isporučuje uređaju promenom širine impulsa u digitalnom signalu (simulacija analognog izlaza primenom digitalnog signala).

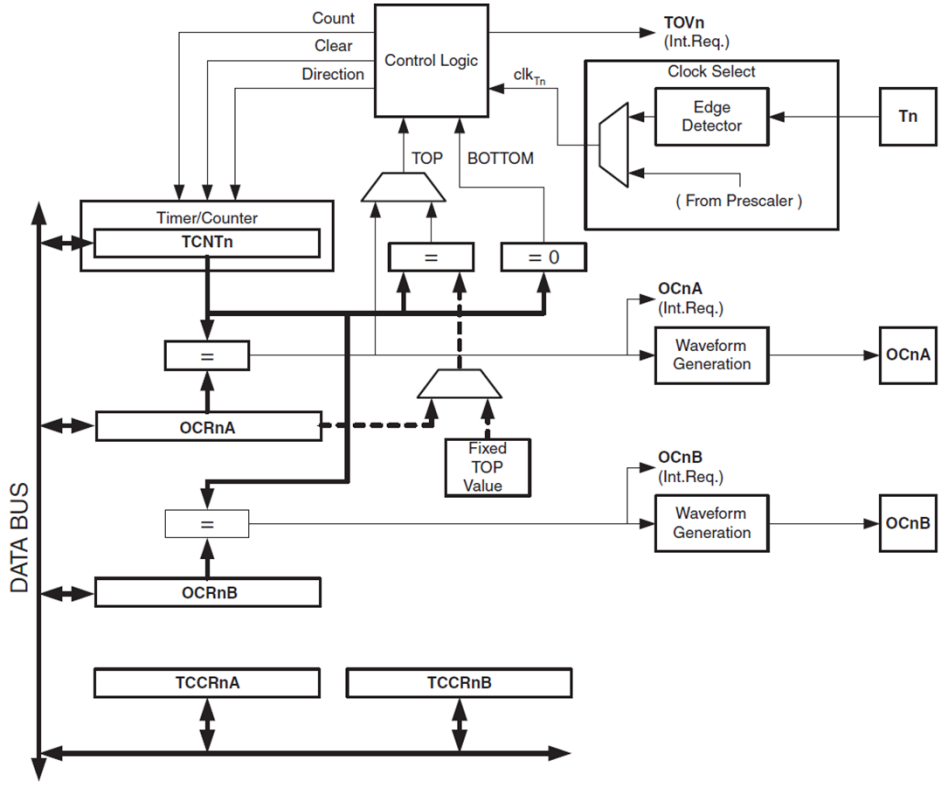


Korišćenje tajmera

- **Osnovna primena** – Zgodno za merenje kratkih vremenskih intervala (do 256 otkucaja takta, ako se koristi 8-bitni registar), ali je merenje vrlo precizno. U datom primeru može se meriti vreme do 16μs, sa preciznošću od 62.5ns.
- **Korišćenje preskalera** – Preskaler je elektronski uređaj koji smanjuje frekvenciju deljenjem nekim predefinisanim faktorom. Produžava se vreme koje se može meriti, ali se smanjuje preciznost. Npr. za N = 1024, može se meriti period od 16384μs (~16.4ms), ali sa rezolucijom od 64μs.
- **Korišćenje pomoćnog registra** – Za produženje vremena uz zadržavanje preciznosti. Kada tajmer dostigne maksimalnu vrednost, on se resetuje na 0. To je događaj koji se može signalizirati prekidom. U ISR možemo inkrementirati neki drugi registar, koji će nam služiti kao bajt veće težine brojača. U našem primeru, za N = 1, može se meriti period od 4096μs (~4ms), ali sa rezolucijom od 62.5ns, do maksimalno 4194304μs (~4.2s) sa rezolucijom od 64μs, za N = 1024.



Blok dijagram 8-bitnog tajmera/brojača



- **Tn** – eksterni izvor takta
- **TCNTn** – tajmer/brojač
- **OCRnA** i **OCRnB** – Output Compare Registri (stalno se porede sa TCNTn i rezultat koristi Waveform Generator da generiše PWM ili izlaz promenljive frekvence na OC pinovima)
- **OCnA** i **OCnB** – Output Compare pinovi
- **TCCRnA** i **TCCRnB** – tajmer/brojač upravljački registri, određuju režim rada (*waveform generation mode*), izlazni mod (*compare output mode*), izvor takta i faktor preskalera.

Režimi rada

- **Normal mode (WGM02:0 = 0) –** Brojač broji od 0 do maksimuma (TOP = 0xFF za 8-bitni, 0xFFFF za 16-bitni), kada pređe maksimum generiše se prekoračenje (*overflow*) i može generisati **TIMERn_OVF_vect** prekid.
- **CTC (Clear Timer on Compere) (WGM02:0 = 2)** – Tajmer se resetuje kada je **TCNTn == OCRnA**. OCRnA definiše **period**. Koristi se za tačno merenje veremena i generisanje signala fiksne frekvencije (ton generator).
- **Fast PWM (WGM02:0 = 3 ili 7)** – Tajmer broji u jednom smeru, od 0 do maksimalne vrednosti (recimo 255 kod 8-bitnog tajmera), nakon čega se resetuje. Kada je **TCNTn == OCRnX** izlazni pin OCnX se menja (npr. prelazi sa visokog na niski nivo). OCRnX određuje **dužinu trajanja impulsa** , tj. **duty cycle**. Formira asimetrični talasni oblik, promena stanja OCnX se vrši **jednom po ciklusu**.

Režimi rada

- **Phase Correct PWM (WGM02:0 = 1 ili 5)** – broju u oba smera (od 0 do 255, pa ponovo do 0), signal je simetričan, manje izobličenje pri upravljanju motorima i D/A, niža frekvencija (duplo više koraka), OCRnX se poredi sa TCNTn pri rastućem i opadajućem brojanju, promena stanja OCnX se dešava **dva puta po ciklusu**.
- **Phase and Frequency Correct PWM** – Slično kao Phase Correct PWM, ali postoji samo kod 16-bitnih tajmera i TOP vrednost dolazi iz ICRn ili OCRnA.
- **Fast PWM sa promenljivim TOP** – Slično kao Fast PWM, ali postoji samo kod 16-bitnih tajmera, a TOP vrednost se definiše softverski (ICRn ili OCRnA).

Registri

TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 16-8. Waveform Generation Mode Bit Description

Mode	WGM2	WGM1	WGM0	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on[1][2]
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	TOP	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Table 16-2. Compare Output Mode, non-PWM Mode

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected
0	1	Toggle OC0A on Compare Match
1	0	Clear OC0A on Compare Match
1	1	Set OC0A on Compare Match

Table 16-3. Compare Output Mode, Fast PWM Mode[1]

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected WGM02 = 1: Toggle OC0A on Compare Match
1	0	Clear OC0A on Compare Match, set OC0A at BOTTOM (non-inverting mode)
1	1	Set OC0A on Compare Match, clear OC0A at BOTTOM (inverting mode)

Table 16-4. Compare Output Mode, Phase Correct PWM Mode[1]

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected WGM02 = 1: Toggle OC0A on Compare Match
1	0	Clear OC0A on Compare Match when up-counting. Set OC0A on Compare Match when down-counting
1	1	Set OC0A on Compare Match when up-counting. Clear OC0A on Compare Match when down-counting

Registri

TCCR0B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TCNT0 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
0x26 (0x46)	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

OCR0A – Output Compare Register A

Bit	7	6	5	4	3	2	1	0	
0x27 (0x47)	OCR0A[7:0]								OCR0A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

OCR0B – Output Compare Register B

Bit	7	6	5	4	3	2	1	0	
0x28 (0x48)	OCR0B[7:0]								OCR0B
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 16-9. Clock Select Bit Description

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$/(No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge
1	1	1	External clock source on T0 pin. Clock on rising edge

FOC0A i **FOC0B** u registru TCCR0B služe za forsiranje (ručno izazivanje) "*Output Compare Match*"-a kada se koristi ne-PWM mod, kao što je CTC (Clear Timer on Compare) ili Normal mod.

Registri

TIMSK0 – Timer/Counter Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6E)	–	–	–	–	–	OCIE0B	OCIE0A	TOIE0	TIMSK0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TIFR0 – Timer/Counter 0 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x15 (0x35)	–	–	–	–	–	OCF0B	OCF0A	TOV0	TIFR0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Kada je **OCIE0B** bit u TIMSK0 registru postavljen (uz I-bit u Status registru) **Timer/Counter0 Compare Match B interapt** je omogućen. Kada se prekid desi (poklope se vrednosti u OCR0B i TCNT0), postavlja se **OCF0B** u TIFR0 registru. OCF0B se resetuje hardverski kada se izvrši odgovarajući vektor za obradu prekida.

Slično važi za **OCIE0A**.

Kada je **TOIE0** bit u TIMSK0 registru postavljen (uz I-bit u Status registru) **Timer/Counter0 Overflow** je omogućen. Kada se desi, tj. javi prekoračenje u Timer/Counter0, postavlja se **TOV0** u TIFR0 registru. TOV0 se resetuje hardverski kada se izvrši odgovarajući vektor za obradu prekida.

Primer 1

```
#include <avr/io.h>
#include <avr/interrupt.h>

void timer0_init_normal(void) {
   TCCR0A = 0;                    // Normal mode (WGM01:0 = 00)
   TCCR0B = (1 << CS02) | (1 << CS00);   // Preskaler = 1024 → 16MHz / 1024 = 15625Hz
   TIMSK0 = (1 << TOIE0);         // Dozvoli overflow prekid
   TCNT0 = 0;                     // Početna vrednost tajmera
   sei();                         // Globalno uključi prekide
}

volatile uint16_t brojac = 0;

ISR(TIMER0_OVF_vect) {
   brojac++;                      // 64µs *256 = 16.384ms
   if (brojac >= 61) {            // 16.381ms * 61 = 0.999424s
      PORTB ^= (1 << PB7);  // Toggle LED na PB7
      brojac = 0;
   }
}

int main(void) {
   DDRB |= (1 << PB7);    // PB7 kao izlaz
   timer0_init_normal();

   while (1) {}
}
```

TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TCCR0B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TIMSK0 – Timer/Counter Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6E)	–	–	–	–	–	OCIE0B	OCIE0A	TOIE0	TIMSK0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

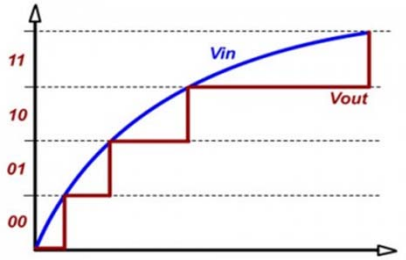
Primer 2

```
#include <avr/io.h>
#include <avr/interrupt.h>

void timer1_init_normal(void) {
   TCCR1A = 0;                    // Normal mod (WGM13:0 = 0000)
   TCCR1B = (1 << CS12);          // Preskaler = 256 (CS12:CS10 = 100)
   TIMSK1 = (1 << TOIE1);         // Dozvoli overflow interrupt
   TCNT1 = 0;                     // Početna vrednost tajmera
   sei();                         // Globalno uključi prekide
}

ISR(TIMER1_OVF_vect) {            // Prekid se dešava na svakih ~1.05 s
    PORTB ^= (1 << PB7);          // Togle pin (npr. LED na PB7)
}

int main(void) {
   DDRB |= (1 << PB7);           // PB7 kao izlaz (LED)
   timer1_init_normal();

   while (1) {}
}
```

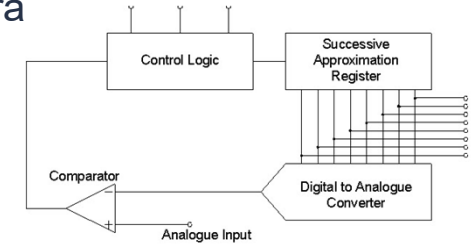
- Timer/Counter0 se koristi za delay(), millis() i micros().
- Ako se koristi za nešto drugo, može se pokvariti rad ovih funkcija.
- Ako koristimo ove funkcije, preporučljivo je koristiti Timer/Counter2, koji je takođe 8-bitni.
- Ako je potrebna veća preciznost, koristiti 16-bitne brojače (Timer/Counter1/3/4/5).
- Za vremenske periode do 4s, nije potrebno koristiti dodatni registar za 16-bitne brojače (kod je jednostavniji).

Analogno-digitalni konvertor

- Analogno-digitalni konvertor (ADC, A/D) je elektronsko kolo koje pretvara analogni signal, kao što je napon ili struja, u digitalni oblik.
- Glavne karakteristike A/D konvertora su brzina uzorkovanja i rezolucija.
- ATmega2560 koristi 10-bitni A/C sa sukcesivnom aproksimacijom (SAR).

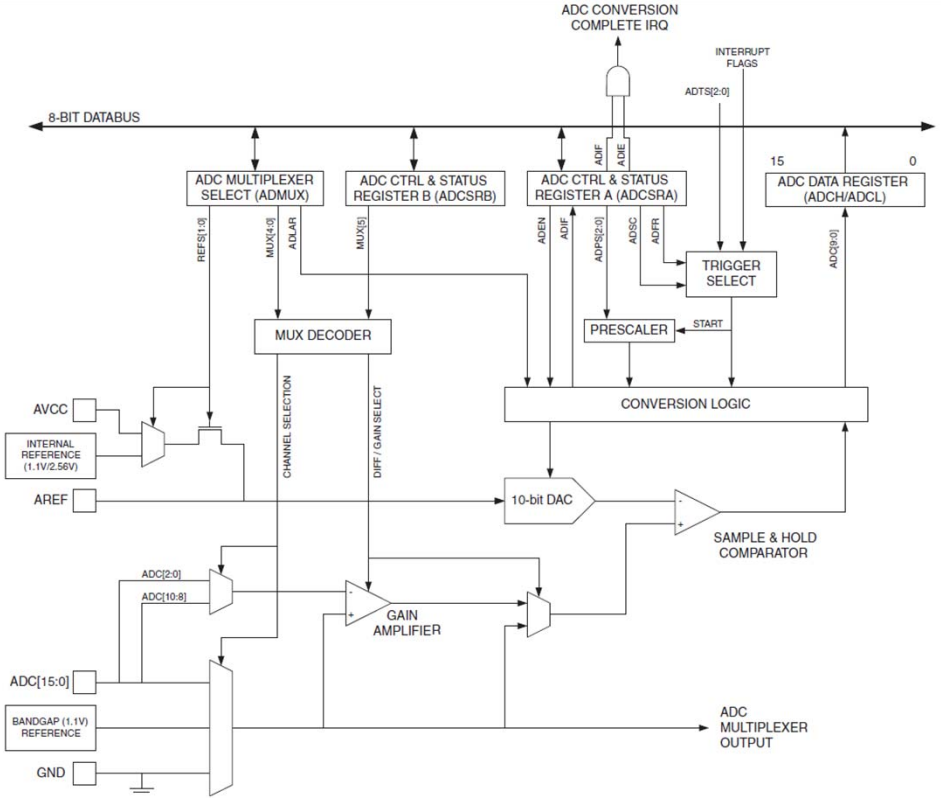


- Successive Approximation ADC (SAR ADC) radi tako što bira bit po bit, počevši od najvišeg (MSB), da bi pronašao digitalnu vrednost koja najbliže odgovara ulaznom analognom naponu.
  - Postavi se jedan bit u interni DAC i uporedi sa ulaznim naponom.
  - Ako je DAC manji bit ostaje 1; ako je veći bit se poništava.
  - Proces se ponavlja za svaki sledeći niži bit, dok se ne dobije konačan 10-bitni rezultat.

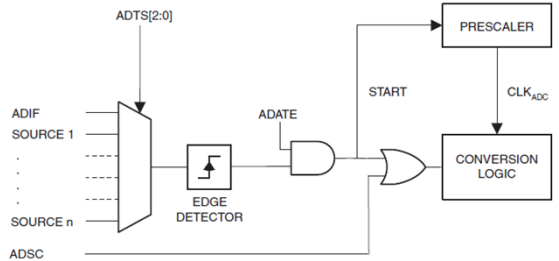


- To je efikasan i brz metod sa fiksnim brojem koraka (npr. 10 za 10-bitni ADC).

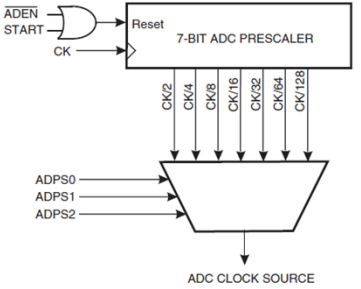
Blok dijagram ADC



ADC Auto Trigger Logic



ADC Prescaler



Registri

ADMUX – ADC Multiplexer Selection Register

Bit	7	6	5	4	3	2	1	0	
(0x7C)	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ADCSRB – ADC Control and Status Register B

Bit	7	6	5	4	3	2	1	0	
(0x7B)	–	ACME	–	–	MUX5	ADTS2	ADTS1	ADTS0	ADCSRB
Read/Write	R	R/W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ADCSRA – ADC Control and Status Register A

Bit	7	6	5	4	3	2	1	0	
(0x7A)	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ADCL and ADCH – The ADC Data Register

Bit	15	14	13	12	11	10	9	8	
(0x79)	–	–	–	–	–	–	ADC9	ADC8	ADCH
(0x78)	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0	ADCL
Bit	15	14	13	12	11	10	9	8	
(0x79)	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADCH
(0x78)	ADC1	ADC0	–	–	–	–	–	–	ADCL
	7	6	5	4	3	2	1	0	

Table 26-4. Input Channel Selections

MUX5:0	Single Ended Input
000000	ADC0
000001	ADC1
000010	ADC2
000011	ADC3
000100	ADC4
000101	ADC5
000110	ADC6
000111	ADC7

Table 26-3. Voltage Reference Selections for ADC

REFS1	REFS0	Voltage Reference Selection[1]
0	0	AREF, Internal V_{REF} turned off
0	1	AVCC with external capacitor at AREF pin
1	0	Internal 1.1V Voltage Reference with external capacitor at AREF pin
1	1	Internal 2.56V Voltage Reference with external capacitor at AREF pin

Table 26-5. ADC Prescaler Selections

ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

ADEN: ADC Enable
ADSC: ADC Start Conversion
ADATE: ADC Auto Trigger Enable (A/D pretvarač će započeti konverziju na pozitivnoj ivici odabranog okidačkog signala.)
ADIF: ADC Interrupt Flag
ADIE: ADC Interrupt Enable
ADLAR: ADC Left Adjust Result

ADLAR = 0

ADLAR = 1

Primer inicijalizacije i čitanja (asm)

```
; ----------------------------------------
; ADC inicijalizacija (jednokratno)
; ----------------------------------------

ldi r16, (1 << REFS0)      ; Referentni napon = AVcc (bit REFS0 = 1, REFS1 = 0)
sts ADMUX, r16             ; Upis u ADMUX (ADC Multiplexer Selection Register)

ldi r16, (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) ; Uključi ADC + preskaler 64
sts ADCSRA, r16            ; Upis u ADCSRA (ADC Control and Status Register A)

; ----------------------------------------
; Pokretanje konverzije i čekanje
; ----------------------------------------

ldi r16, (1 << ADSC)       ; Pokreni konverziju (bit ADSC)
sts ADCSRA, r16

loop_wait:
lds r17, ADCSRA
sbrs r17, ADSC             ; Ako je ADSC = 0, konverzija je gotova i preskače se sledeća instrukcija
rjmp loop_wait             ; Inače čekaj
```

ADMUX – ADC Multiplexer Selection Register

Bit	7	6	5	4	3	2	1	0	
(0x7C)	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX

ADCSRA – ADC Control and Status Register A

Bit	7	6	5	4	3	2	1	0	
(0x7A)	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA

ADCSRA – ADC Control and Status Register A

Bit	7	6	5	4	3	2	1	0	
(0x7A)	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA

Primer inicijalizacije i čitanja (asm)

```
; -------------------------------------
; Čitanje rezultata iz ADCL i ADCH
; -------------------------------------

lds r18, ADCL          ; Prvo se čita ADCL!
lds r19, ADCH          ; Zatim ADCH

; -------------------------------------
; adc rezultat: r19:r18 (10-bitni)
; -------------------------------------

; Primer: čuvanje u SRAM promenljive (opciono)
sts adc_low, r18
sts adc_high, r19

; Dalja obrada...
```

```
; -------------------------------------
; Definisanje prostora za rezultat
; -------------------------------------

.def adc_l = r18
.def adc_h = r19

.dseg
.org 0x0100
adc_low:  .byte 1
adc_high: .byte 1
```

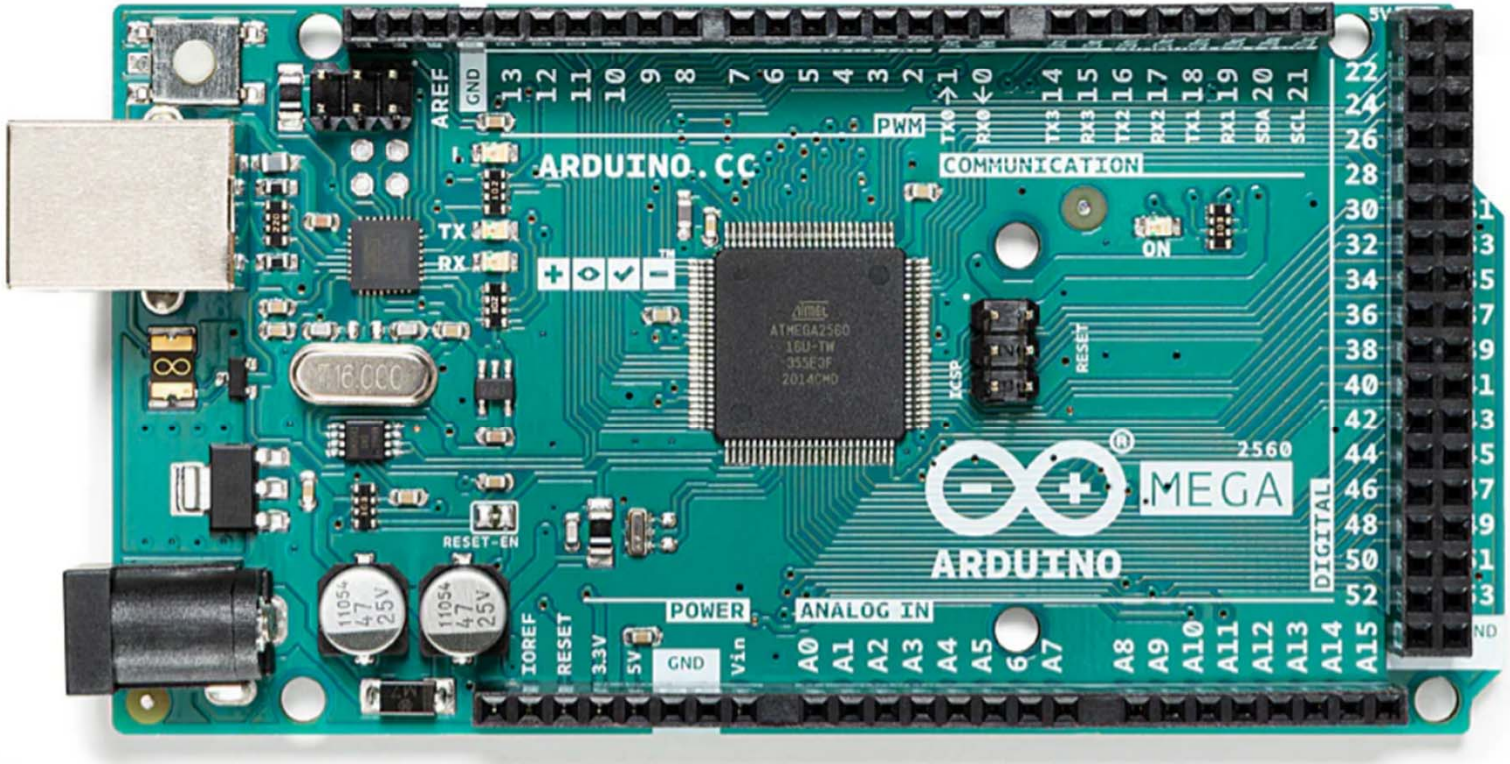
Primer inicijalizacije i čitanja (C)

```c
#include <avr/io.h>

void adc_init(void) {
    ADMUX = (1 << REFS0);    // Referentni napon = AVcc (bit REFS0 = 1), ulazni kanal = ADC0 (MUX = 0000)
    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1);    // Preskaler = 64 (ADPS2 = 1, ADPS1 = 1), enable ADC (ADEN = 1)
}

uint16_t adc_read(uint8_t channel) {
   ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);  // Očistimo prethodni izbor kanala, tj. donjih 4 bita (kanali 0-15), i postavimo željeni
   ADCSRA |= (1 << ADSC);         // Pokreni konverziju (ADSC = 1)

   while (ADCSRA & (1 << ADSC)); // Čekaj da se završi (ADSC = 0)

   uint16_t rezultat = ADC;         // Makro ADC čita i ADCL i ADCH, tj. vraća ADCL + (ADCH<<8)
   return rezultat;                 // 0–1023
}

int main(void) {
   adc_init();
   uint16_t vrednost = adc_read(0);  // Očitava analognu vrednost sa ulaza A0 (ADC0)
   while (1) {}
}
```

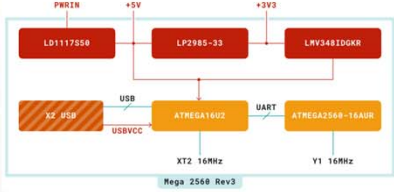
Literatura

- **ATmega640/V-1280/V-1281/V-2560/V-2561/V** [DATASHEET] **2549Q–AVR–02/2014** (atmel-2549-8-bit-avr-microcontroller-atmega640-1280-1281-2560-2561_datasheet.pdf)
- **Arduino® Mega 2560 Rev3** User Manual, SKU: A000067 (A000067-datasheet.pdf)

Arduino Mega 2560



ATmega16U2
ATmega2560



In-Circuit Serial Programming (ICSP)

AVR MKII

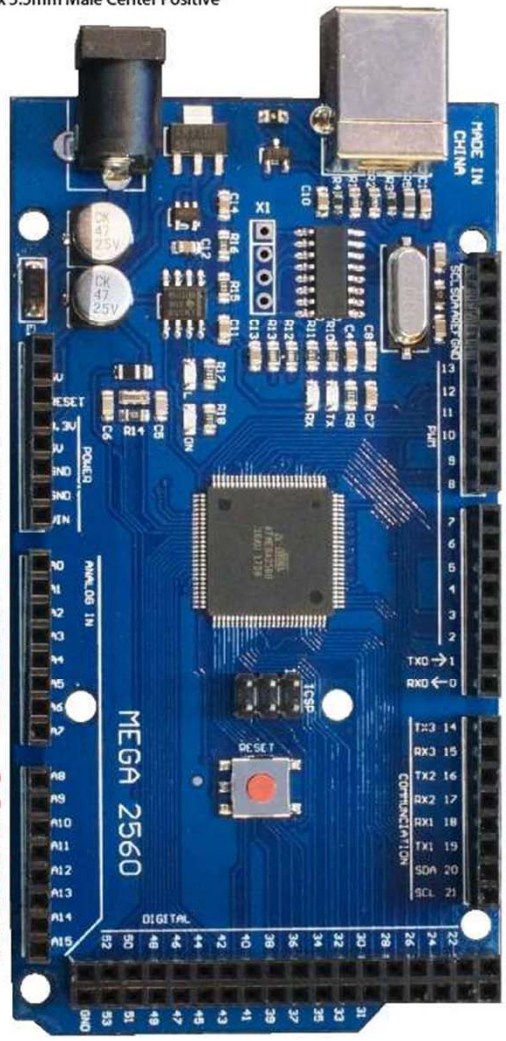
JTAG ICE3

- ICSP omogućuje upload koda ili bootloader-a na mikrokontroler kada je on već ugrađen na ploču.
- ICSP često se koristi hardverske SPI pinove mikrokontrolera za komunikaciju.
- Koristi se kada USB port na ploči ne radi ili treba ažurirati bootloader.

Mega 2560 Pinout

DC Power Jack 7-12VDC Input
2.1mm x 5.5mm Male Center Positive

USB-B Port To Computer



Red numbers in parenthesis are the name to use when referencing that pin.
Analog pins are referenced as A0 thru A15 even when using as digital I/O

No Connection
I/O Reference Voltage for shields
Reset Input
3.3V Output @ 50mA
5V Output or Input
Ground
Ground
7-12V Output or Input

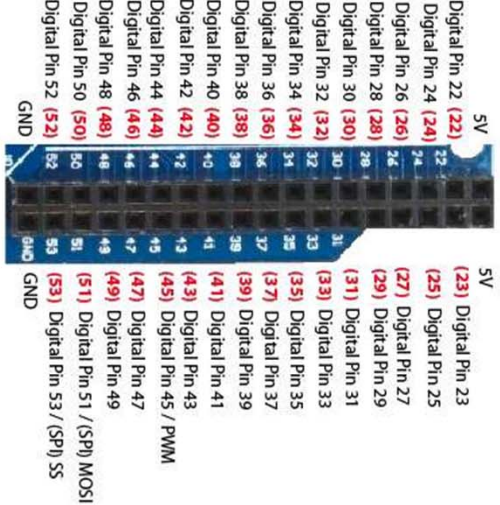
Analog Pin 0 / Digital Pin 54 **(A0)**
Analog Pin 1 / Digital Pin 55 **(A1)**
Analog Pin 2 / Digital Pin 56 **(A2)**
Analog Pin 3 / Digital Pin 57 **(A3)**
Analog Pin 4 / Digital Pin 58 **(A4)**
Analog Pin 5 / Digital Pin 59 **(A5)**
Analog Pin 6 / Digital Pin 60 **(A6)**
Analog Pin 7 / Digital Pin 61 **(A7)**

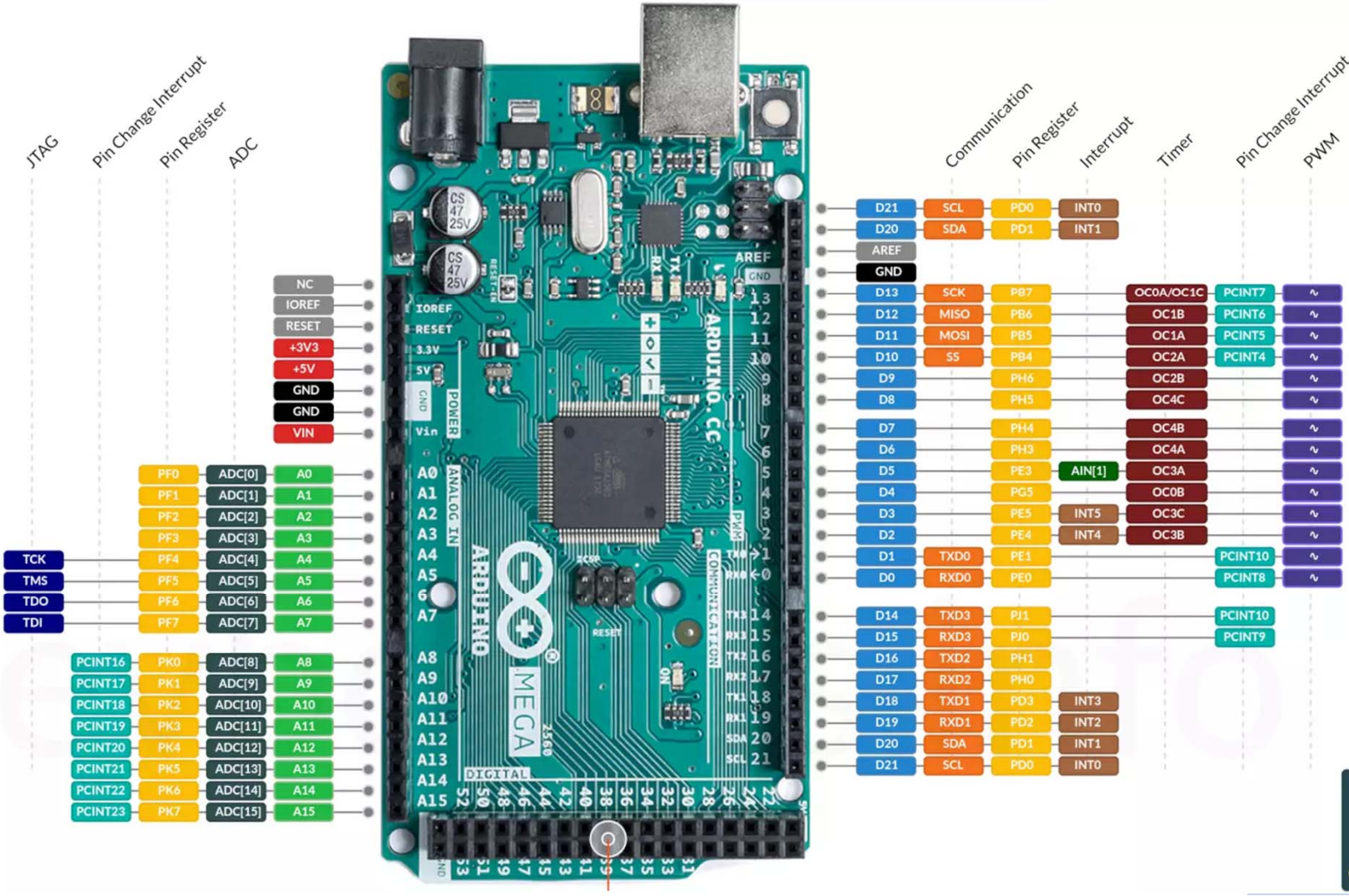
Analog Pin 8 / Digital Pin 62 **(A8)**
Analog Pin 9 / Digital Pin 63 **(A9)**
Analog Pin 10 / Digital Pin 64 **(A10)**
Analog Pin 11 / Digital Pin 65 **(A11)**
Analog Pin 12 / Digital Pin 66 **(A12)**
Analog Pin 13 / Digital Pin 67 **(A13)**
Analog Pin 14 / Digital Pin 68 **(A14)**
Analog Pin 15 / Digital Pin 69 **(A15)**

(I2C) SCL - Serial Clock
(I2C) SDA - Serial Data
Analog Reference Voltage
Ground
(13) Digital Pin 13 / PWM / Connected to on-board LED
(12) Digital Pin 12 / PWM
(11) Digital Pin 11 / PWM
(10) Digital Pin 10 / PWM
(9) Digital Pin 9 / PWM
(8) Digital Pin 8 / PWM

(7) Digital Pin 7 / PWM
(6) Digital Pin 6 / PWM
(5) Digital Pin 5 / PWM
(4) Digital Pin 4 / PWM
(3) Digital Pin 3 / PWM / Ext Int 5
(2) Digital Pin 2 / PWM / Ext Int 4
(1) Digital Pin 1 / Serial Port 0 TXD (Main Serial Port)
(0) Digital Pin 0 / Serial Port 0 RXD (Main Serial Port)

(14) Digital Pin 14 / Serial Port 3 TXD
(15) Digital Pin 15 / Serial Port 3 RXD
(16) Digital Pin 16 / Serial Port 2 TXD
(17) Digital Pin 17 / Serial Port 2 RXD
(18) Digital Pin 18 / Serial Port 1 TXD / Ext Int 3
(19) Digital Pin 19 / Serial Port 1 RXD / Ext Int 2
(20) Digital Pin 20 / (I2C) SDA / Ext Int 1
(21) Digital Pin 21 / (I2C) SCL / Ext Int 0




JTAG
Pin Change Interrupt
Pin Register
ADC
Communication
Pin Register
Interrupt
Timer
Pin Change Interrupt
PWM
NC
IOREF
RESET
+3V3
+5V
GND
GND
VIN
TCK
TMS
TDO
TDI
PF0 ADC[0] A0
PF1 ADC[1] A1
PF2 ADC[2] A2
PF3 ADC[3] A3
PF4 ADC[4] A4
PF5 ADC[5] A5
PF6 ADC[6] A6
PF7 ADC[7] A7
PCINT16 PK0 ADC[8] A8
PCINT17 PK1 ADC[9] A9
PCINT18 PK2 ADC[10] A10
PCINT19 PK3 ADC[11] A11
PCINT20 PK4 ADC[12] A12
PCINT21 PK5 ADC[13] A13
PCINT22 PK6 ADC[14] A14
PCINT23 PK7 ADC[15] A15
D21 SCL PD0 INT0
D20 SDA PD1 INT1
AREF
GND
D13 SCK PB7 OC0A/OC1C PCINT7 ~
D12 MISO PB6 OC1B PCINT6 ~
D11 MOSI PB5 OC1A PCINT5 ~
D10 SS PB4 OC2A PCINT4 ~
D9 PH6 OC2B ~
D8 PH5 OC4C ~
D7 PH4 OC4B ~
D6 PH3 OC4A ~
D5 PE3 AIN[1] OC3A ~
D4 PG5 OC0B ~
D3 PE5 INT5 OC3C ~
D2 PE4 INT4 OC3B ~
D1 TXD0 PE1 PCINT10 ~
D0 RXD0 PE0 PCINT8 ~
D14 TXD3 PJ1 PCINT10
D15 RXD3 PJ0 PCINT9
D16 TXD2 PH1
D17 RXD2 PH0
D18 TXD1 PD3 INT3
D19 RXD1 PD2 INT2
D20 SDA PD1 INT1
D21 SCL PD0 INT0
ARDUINO.CC
ARDUINO
MEGA
2560
DIGITAL
POWER
ANALOG IN
COMMUNICATION
devboards.info

Ključni pojmovi

- Portovi
- Brojači i tajmeri
- A/D i D/A (PWM) konverzija
- Arhitektura Arduino Mega 2560
- ICSP

PITANJA